Object Recognition and Sorting Conveyor Belt System

A machine-vision conveyor system that detects **red, green, and blue blocks** using an **ESP32-CAM** running an **Edge Impulse** model, then sorts them using an **Arduino Uno** that controls a **NEMA17 stepper motor** through an **A4988 driver** and an **SG90 servo motor**.

This project is designed as a compact mechatronics and embedded-systems build for demonstration, competition, or academic prototyping.

Table of Contents

- Overview
- System Features
- Architecture
- Hardware Components
- Power Supply Design
- Wiring and Interfacing
- Mechanical Arrangement
- Control Logic
- Software Design
- Edge Impulse Workflow
- Arduino and ESP32 Communication
- Project Structure
- Assembly Notes
- Troubleshooting
- Future Improvements
- License

Overview

The system uses a conveyor belt to move colored blocks into a detection zone. An **ESP32-CAM** mounted above the belt captures images and classifies the object color in real time. The classification result is sent to an **Arduino Uno**, which decides how to move the **servo-based sorting gate** and how to run the conveyor motor.

The conveyor is driven by a **single NEMA17 stepper motor** through an **A4988 stepper driver**. Two **IR sensor modules** are used for belt start and end detection. The system is powered from a **single 12V supply**, with regulated 5V available for low-voltage electronics.

This build intentionally **does not use a CNC shield**, because only one stepper axis is required.

System Features

- Real-time color recognition for **red**, **green**, and **blue** blocks
- ESP32-CAM vision processing using **Edge Impulse**
- Arduino Uno-based actuator control
- Conveyor belt driven by **NEMA17 + A4988**
- Servo-based sorting gate using **SG90**
- IR start and end sensors for conveyor flow control
- Single 12V power input with regulated 5V logic rail
- Modular design for classroom demonstration and rapid prototyping

---

Architecture

High-Level Block Flow

Input: Colored block placed on conveyor

Sensing:

- IR sensor detects object presence
- ESP32-CAM captures image
- Edge Impulse model classifies color

Control:

- ESP32-CAM sends classification result to Arduino Uno
- Arduino Uno manages stepper and servo actions

Output:

- Stepper motor moves conveyor belt
- Servo deflects object to the correct bin

Main Roles

- **ESP32-CAM**: image capture and classification only
- **Arduino Uno**: conveyor motion and sorting logic
- **A4988**: stepper driver
- **NEMA17**: conveyor drive motor
- **SG90**: sorting actuator
- **IR sensors**: object detection at key points

Hardware Components

Required Components

- 1 × ESP32-CAM module
- 1 × Arduino Uno
- 1 × NEMA17 stepper motor
- 1 × A4988 stepper driver
- 1 × SG90 servo motor
- 2 × IR sensor modules
- 1 × 12V DC power supply
- 1 × conveyor belt material
- Bearings, shafts, nuts, bolts, washers, frame materials
- Motor coupling
- Jumper wires and breadboard or perfboard
- Capacitors for power smoothing
- 5V buck converter or stable 5V regulator

Not Used

- CNC Shield

Power Supply Design

The system uses one **12V DC supply** as the main source.

Power Distribution

- **12V rail**
- A4988 motor supply (VMOT)
- NEMA17 stepper motor

- Optional input to Arduino Uno VIN

- **5V rail**

- ESP32-CAM
- SG90 servo
- IR sensors
- Arduino Uno logic (if using regulated 5V input)

Recommended Approach

Use a **12V to 5V buck converter** for a stable logic supply. This is preferred because the servo and ESP32-CAM can draw current spikes that may reset weak regulators.

Important Rules

- All grounds must be connected together
- Place a large capacitor near the A4988 motor supply pins
- Keep motor wiring separate from signal wiring where possible
- Do not power the servo directly from an unstable rail

Wiring and Interfacing

ESP32-CAM to Arduino Uno

Use **UART serial communication**.

- ESP32-CAM TX → Arduino Uno RX
- ESP32-CAM GND → Arduino GND
- Shared common ground is mandatory

The ESP32-CAM sends class labels such as:

- `RED`
- `GREEN`
- `BLUE`

Arduino Uno to A4988

- STEP pin → Arduino digital output
- DIR pin → Arduino digital output
- EN pin → Arduino digital output or tied active-low
- VDD → 5V
- GND → common ground
- VMOT → 12V
- Motor coils → NEMA17 phases

Arduino Uno to Servo

- Servo signal → PWM-capable Arduino pin
- Servo VCC → 5V
- Servo GND → common ground

Arduino Uno to IR Sensors

- Start IR output → Arduino input pin
- End IR output → Arduino input pin
- VCC → 5V
- GND → common ground

Mechanical Arrangement

Conveyor Layout

1. **Loading zone**
2. User places object here
3. Start IR sensor detects presence
4. **Detection zone**
5. ESP32-CAM mounted above the belt
6. Camera sees one object at a time
7. **Sorting zone**
8. Servo gate positioned at a fixed point
9. Object is diverted based on detected color
10. **Exit zone**
11. End IR sensor detects object leaving
12. Belt stops if no new object is present

Belt Motion

The NEMA17 rotates a pulley or roller to move the conveyor belt. The belt should travel smoothly and at a constant speed so the camera can classify the object consistently.

Control Logic

Idle State

- Conveyor is stopped
- Servo stays in neutral position
- Arduino waits for the start IR sensor

Start Trigger

When the start IR sensor detects an object:

- Arduino enables the A4988 driver
- Conveyor begins moving
- ESP32-CAM starts classification if not already active

Classification

ESP32-CAM:

- captures the object image
- runs Edge Impulse inference
- identifies `red`, `green`, or `blue`
- sends the result to Arduino

Sorting

Arduino:

- receives the label
- moves servo to the matching sorting angle
- waits for the object to pass
- returns servo to neutral

Stop Condition

When the end IR sensor detects that the object has left and no new object is present:

- Arduino stops the stepper pulses
- conveyor returns to idle

If a new object is already waiting at the start sensor, the conveyor continues running.

Software Design

ESP32-CAM Firmware

The ESP32-CAM program should:

1. initialize the camera
2. initialize serial communication
3. load the Edge Impulse inference library
4. capture frames continuously
5. run classification on each frame

6. send the detected color label to Arduino

ESP32-CAM Output Format

A simple serial message is recommended:

```
RED
GREEN
BLUE
```

Arduino Firmware

The Arduino Uno program should:

1. configure stepper driver pins
2. configure servo output
3. configure IR sensor inputs
4. initialize serial communication
5. read color labels from ESP32-CAM
6. drive the conveyor motor
7. move the servo gate according to the detected class
8. stop the belt when the system is idle

Edge Impulse Workflow

Dataset

Train the model with three classes:

- Red block
- Green block
- Blue block

Data Collection Tips

- Capture images under the same lighting conditions used in the final system
- Include blocks at different positions on the belt
- Include multiple backgrounds if relevant
- Capture samples with slight angle and brightness variation

Model Goal

The model does not need to detect arbitrary objects. It only needs to classify the three supported color blocks.

Recommended Output

A lightweight classification model is best because the ESP32-CAM has limited memory and processing resources.

Arduino and ESP32 Communication

Suggested Protocol

The ESP32-CAM sends one line per detection:

````
RED\n
GREEN\n
BLUE\n```

### Arduino Action Mapping

- `RED` → servo angle for red bin
- `GREEN` → servo angle for green bin
- `BLUE` → servo angle for blue bin

### Best Practice

Use a confidence threshold so the system only acts on reliable predictions.

Example logic:

- if confidence is too low, ignore the prediction
- if confidence is high, send the label to Arduino

---

## Project Structure

A clean repository layout may look like this:

```text

project-root/
├── esp32_cam/
````
````

```
│   ├── esp32_cam_edge_impulse.ino
│   └── camera_config.h
├── arduino_uno/
│   ├── conveyor_sorting_controller.ino
│   └── motor_control.h
├── docs/
│   ├── wiring_diagram.png
│   ├── architecture_diagram.png
│   ├── bill_of_materials.md
│   └── report.pdf
├── mechanical/
│   ├── frame_design/
│   └── belt_assembly/
└── README.md
```

Assembly Notes

- Mount the ESP32-CAM directly above the detection zone
- Keep the camera aligned so each object appears centered in the frame
- Adjust IR sensor sensitivity carefully
- Tune the servo angles based on physical bin placement
- Set the A4988 current limit properly for the NEMA17 motor
- Test motor movement before enabling camera-based sorting
- Verify that the 5V rail can handle the ESP32-CAM and servo together

---

Troubleshooting

Conveyor does not move

- Check A4988 wiring
- Verify motor power at 12V
- Check ENABLE pin state
- Confirm step and direction signals from Arduino

Servo jitters or resets

- Improve 5V supply stability
- Use a separate buck converter for servo power
- Add decoupling capacitors
- Make sure grounds are common

ESP32-CAM does not classify reliably

- Improve lighting

- Rebuild the Edge Impulse dataset
- Increase training samples
- Check camera focus and mounting angle

Arduino does not receive labels

- Check serial baud rate
- Verify TX/RX wiring
- Confirm common ground
- Make sure the ESP32-CAM is actually sending class text

False sorting

- Increase confidence threshold
- Reduce conveyor speed
- Improve camera framing
- Train with more varied samples

Future Improvements

- Add multiple sorting bins
- Replace the IR sensors with better object detection sensors
- Add LCD or OLED status display
- Add Wi-Fi logging from the ESP32-CAM
- Implement automatic speed tuning for the conveyor
- Use a dedicated 5V regulator for cleaner servo power
- Add emergency stop and manual override buttons

License

This project can be licensed under any license you prefer, such as MIT or Apache 2.0, depending on whether it is intended for public release or internal academic use.

Credits

Concept inspired by conveyor-based machine-vision sorting systems and adapted for a simplified single-stepper, single-servo implementation.